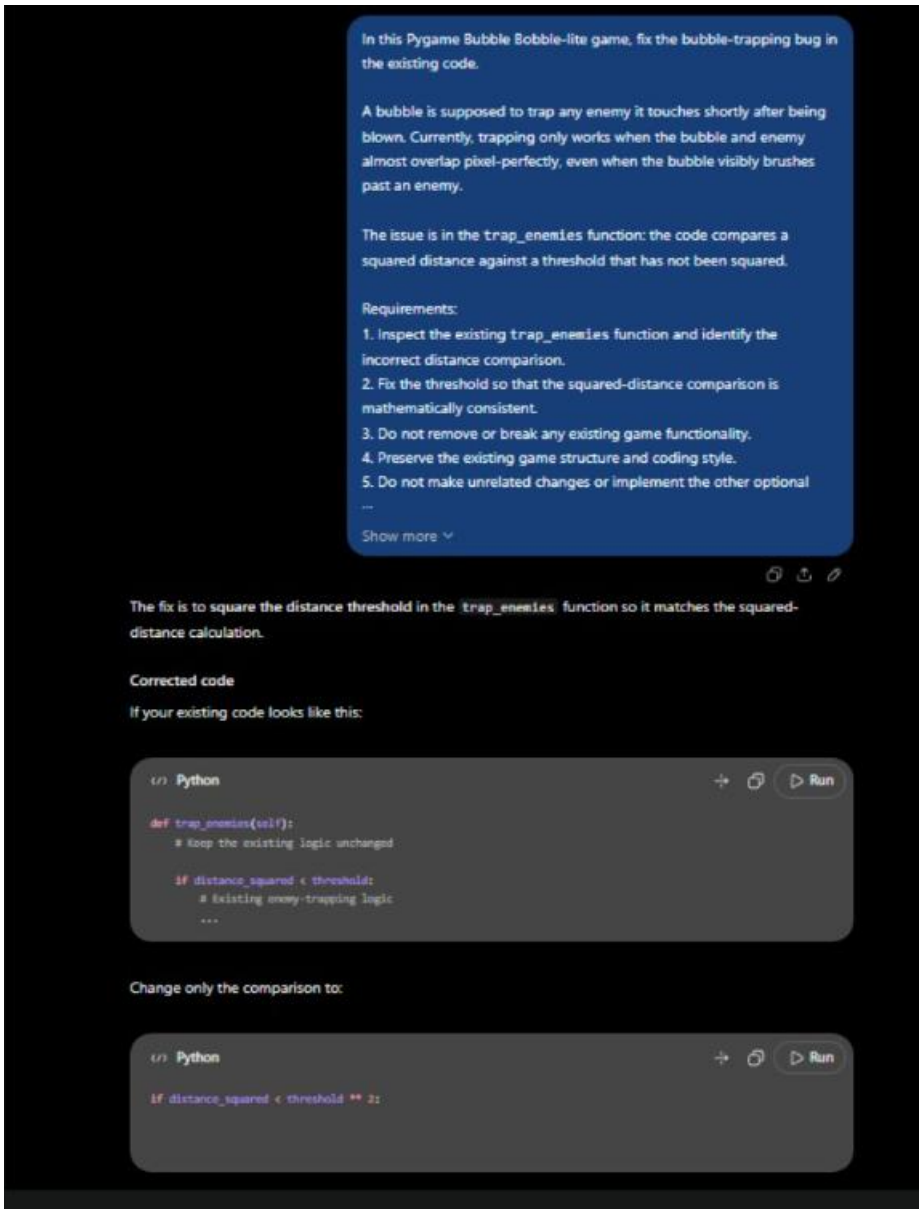
Software Engineering Lab:4

Name : P. Chinni Krishna Reddy

SRN: PES1UG24AM209



Now implement the second task in the same Pygame Bubble Bobble game.
Complete the `bubble_tint(bubble)` function. It receives a `Bubble` object and must return an RGB color tuple or `None`.
Requirements:

1. Use `bubble.enemy` to distinguish trapped bubbles from ordinary bubbles.
2. Use `bubble.life` to change the color as a trapped bubble approaches expiration.
3. Use a clear color progression, such as pink for a trapped bubble with plenty of time, orange when it is getting close to expiring, and red when it is about to expire.
4. Return `None` for ordinary bubbles so the game's existing default color is preserved.
5. Do not break existing drawing or gameplay functionality.
6. Explain the implementation.

Do not implement the remaining two tasks yet.

For Task 2, complete `bubble_tint(bubble)` by checking whether the bubble contains an enemy and using its remaining `life` to determine the color. This is a small, isolated change that preserves the existing gameplay and drawing logic.

1. Corrected `bubble_tint(bubble)` function

Replace the existing function with:

Python

Run

```python
def bubble_tint(bubble):
    if bubble.enemy is None:
        return None

    if bubble.life > 60:
        return (255, 105, 180)  # Pink: plenty of time
    elif bubble.life > 30:
        return (255, 165, 0)    # Orange: nearing expiration
    else:
        return (255, 0, 0)      # Red: about to expire
```

Now implement the third task in the same Pygame Bubble Bobble game.
Complete the `on_fruit_collected(fruit)` function. It is called when the player collects a fruit, after the fruit's points have been added and the fruit has been removed from play.
Requirements:

1. Implement a visible running fruit-collection counter in the HUD.
2. Increment the counter exactly once for each collected fruit.
3. Reset the counter when a new game is started using the existing reset functionality.
4. Preserve the existing fruit scoring and collection behavior.
5. Make the smallest necessary changes to integrate the counter into the game.
6. Explain the changes and identify the relevant code sections.

Do not implement the bonus-life feature yet.

Worked for 13s

Task 3 can be implemented with a small change to the existing `game_bubble_bobble_final.py` file. The version I found already contains the fruit counter logic, HUD display, reset logic, and collection callback. If that is your current file, these sections are already implemented.

1. Fruit counter variable

At the top of the file, keep the global counter:

Python Run

```python
FRUITS_COLLECTED = 0
```

This tracks the total number of fruits collected during the current game.

2. Complete `on_fruit_collected(fruit)`

Use this function:

Python Run

```python
def on_fruit_collected(fruit):
    """Track collected fruit so the total can be shown in the HUD."""
    global FRUITS_COLLECTED
    FRUITS_COLLECTED += 1
```

3. Verify fruit collection

Inside `Game.update()`, the collection logic should remain in this order:

```python
for fruit in self.fruits[:]:
    fruit.update(dt)
    if fruit.rect.colliderect(player.rect):
        self.score += fruit.value
        self.fruits.remove(fruit)
        on_fruit_collected(fruit)
    elif fruit.life <= 0:
        self.fruits.remove(fruit)
```

Why this works: the fruit's points are added and the fruit is removed before the counter increments. Iterating over `self.fruits[:]` also allows fruits to be removed safely while iterating.

Do not add another increment elsewhere; that could count the same collection twice.

4. Reset the counter when a new game starts

Inside `Game.reset()`, preserve this initialization:

```python
def reset(self):
    global FRUITS_COLLECTED
    FRUITS_COLLECTED = 0

    self.level, self.score, self.lives, self.combo, self.state = 1, 0, 3, 0, "play"
    self.bonus_awarded = 0
    self.player = Player()
    self.fruits = []
    self.start_level()
```

Pressing R already calls `game.reset()` in the existing main loop, so the fruit count returns to zero when the game resets.

5. Display the counter in the HUD

Inside `Game.draw()`, retain the existing HUD rendering:

```python
hud = self.font.render(
    f"Score {self.score}  Lives {self.lives}  Level {self.level}  "
    f"Fruits {FRUITS_COLLECTED}  R = reset",
    True,
    (240, 240, 240),
)
screen.blit(hud, (10, 8))
```

Now implement the final task in the same Pygame Bubble Bobble game.
Complete the `bonus_life_threshold()` function.
Requirements:

1. Return `5000` as the score threshold for earning an extra life.
2. Use the existing `self.bonus_awarded` bookkeeping in the game rather than implementing a second bonus-life system.
3. Ensure the existing update logic awards an extra life when the score crosses each new 5,000-point threshold.
4. Do not award the same threshold more than once.
5. Preserve the existing scoring, lives, and reset functionality.
6. Explain how the implementation works.

Do not make unrelated changes.

The final task requires only a small change: ensure `bonus_life_threshold()` returns `5000` and keep the existing `self.bonus_awarded` logic in `Game.update()`.

1. Complete `bonus_life_threshold()`

In `game_bubble_bobble_final.py`, use:

Python Run

```python
def bonus_life_threshold():
    """Return the score required to earn an extra life."""
    return 5000
```

2. Verify the existing bonus-life logic

Inside `Game.update()`, preserve this code:

Python Run

```python
threshold = bonus_life_threshold()
if threshold and self.score // threshold > self.bonus_awarded:
    self.bonus_awarded = self.score // threshold
    self.lives += 1
```